

该项目面向智驾场景下车辆误识别、漏识别等高频问题，传统模式下需要工程师人工归类现象，在百万行级代码库中定位相关模块、跨 Bag 数据和代码做根因推理，单个问题平均耗时 3~5 小时，强依赖资深员工经验，且解决方案难以沉淀复用。因此我主导设计并落地了基于多 Agent 协作的 ADAS 根因分析系统，将“现象归类—数据定位—代码检索—根因分析—方案建议”端到端打通。Triage Agent（问诊分诊智能体），根据故障描述识别问题类型，并结合预先构建的 200+ 业务模块与代码路径映射表，将业务问题转换为候选代码模块和高质量检索 Query，从而提前缩小检索范围；对于分类置信度较低的长尾问题，系统会扩大检索范围，必要时转人工处理。完成任务分诊后，LangGraph 并行调度 Data Agent 和 Code Agent 开展双路取证：Data Agent 负责解析 Bag 数据，定位故障发生的时间窗口、关键传感器信号及异常现象，输出结构化的数据证据；Code Agent 则通过 Code RAG 检索相关源码，其中使用 Tree-sitter 按函数、类粒度切分代码，利用 Jina Code Embeddings 建立代码语义表示，并结合静态分析构建存储在 Neo4j 中的函数调用图谱，通过关键词召回、语义召回和调用链扩展获取候选代码，再经过 RRF 排名融合与 Rerank 精细重排序，输出带有函数位置和代码引用的关键证据。当两条取证分支完成后，RCA Agent（根因分析智能体）综合 Bag 异常数据与代码逻辑，建立故障现象、程序行为和候选根因之间的关联，并要求根因假设同时具备 Bag 证据和具体代码支撑。为避免 LLM 产生虚假引用或缺乏依据的结论，系统引入 Verifier Agent（证据校验智能体），对代码行号、Bag 引用和证据一致性进行真实性校验；如果校验失败，则根据失败原因进行 Retry（重试）或 Replan（重新规划），重新分析或补充取证，无法可靠完成时则转人工处理。通过校验后，Solution Agent（解决方案智能体）根据当前根因分析结果，并结合长期 Memory 中沉淀的历史故障案例、领域知识及处理 Skills，生成修复建议与根因分析报告。与此同时，系统利用 LangGraph State + Checkpoint 维护当前任务的短期工作记忆，并通过长上下文摘要压缩控制推理成本；当工单关闭且处理结果得到确认后，再将故障现象、关键证据、确认根因与解决方案结构化写入长期记忆，形成从故障分析到案例沉淀、再到历史经验复用的完整闭环。根据简历中的项目评测，最终代码模块 Top-5 召回率超过 90%，根因 Top-1 命中率达到 85%，引用错误率由 23% 降至 6%。

ADAS 根因分析系统

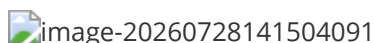
1、项目介绍

该项目面向智驾场景下车辆误识别、漏识别等高频问题，传统模式下需要工程师人工归类现象，在百万行级代码库中定位相关模块、跨 Bag 数据和代码做根因推理，单个问题平均耗时 3~5 小时，强依赖资深员工经验，且解决方案难以沉淀复用。因此我主导设计并落地了基于多 Agent 协作的 ADAS 根因分析系统，将“现象归类—数据定位—代码检索—根因分析—方案建议”端到端打通。

2、Multi-Agent 架构

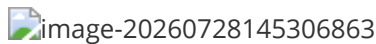
Triage（现象归类与任务路由）

是整个工作流的入口，负责将工程师提交的非结构化工单转换成后续可以直接执行的结构化任务。它主要回答两个问题，一是当前工单属于什么问题类型？，二是后续应该从哪些 Bag 数据和代码模块开始排查，尽可能缩小数据和代码检索范围。首先对工单内容进行标准化，将工程师上传的多源信息转换成统一的现象描述；其次根据现象判断所属的高频问题类型和候选业务模块，完成误识别、漏识别和无识别的分类（项目已经覆盖部门约 80% 的高频问题类型，因此对于常见问题，Triage Agent 可以直接输出相应的分类结果。对于长尾问题，则需要降低置信度或扩大后续检索范围）；同时，项目中建立了覆盖 200 多个业务模块→代码路径的映射表（第一次搜索范围收缩），让上一阶段的分类结果可以直接转化为代码检索的高质量 query（Code Agent 不需要直接在百万行级代码库中进行完全开放式搜索，而是可以先在候选业务模块对应的代码范围内检索）；最后生成两个并行执行的子任务，Data Agent 的数据定位任务和 Code Agent 的代码检索任务。（输入是原始工单，输出标准化的任务描述，为后续提供更高质量的输入）



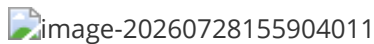
Data (数据定位)

负责在 Bag 数据中定位与当前问题相关的证据，包括发生位置、数据帧、时间段、信号字段以及观察到的异常现象，为 RCA Agent 提供引用信息。这里还做了异常和降级处理，如果 Bag 数据缺失或无法找到足够证据，不应该自行补全或猜测，而应明确返回定位失败及原因并建议人工检查。



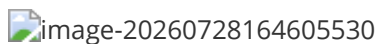
Code (代码检索)

负责在代码库中定位与当前问题相关的证据，包括业务模块、函数或类、具体代码段以及上下游调用关系，高召回地找到候选代码。知识库构建阶段，按函数/类粒度切分代码并使用代码专用嵌入模型 Jina Code Embeddings，同时使用 Tree-sitter 解析代码语法结构，并结合静态分析构建函数级调用图谱存入 Neo4j (故障真正发生的位置不一定在语义上最相似)，实现关键词召回、语义召回和调用链扩展的三路混合检索 (与通用 embedding 相比，召回率提升 31%)。检索阶段，先使用之前生成的高质量 Query 检索找到初始候选，也就是调用链分析的入口节点，随后在 Neo4j 中进一步查找上下游调用函数以及相关依赖，然后通过 RRF 算法融合，最后进行 Rerank。



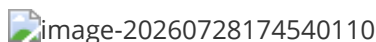
RCA (证据融合与根因推理)

是整个系统的核心推理节点，将原始描述、现象分类、数据与代码证据等信息整合。首先判断 Data Agent 提供的数据证据能否支持工单中的异常现象，随后分析 Bag 数据异常与候选代码逻辑之间是否存在合理关系，最后生成根因候选，并要求每条根因假设必须同时引用 Bag 数据和具体代码。这里还引入了一个新增的 Verifier Agent 校验机制，对引用做真实性校验，引用错误系统自动重新生成。



Solution (历史案例检索与方案建议)

基于根因和相似历史案例 (类似问题是否发生过，之前是如何修复的，当前问题可以复用哪些经验) 生成解决方案建议。将系统中已关闭工单的结案数据自动结构化入库，包括问题现象、涉及业务模块、最终根因、修复 commit，然后根据当前问题构造检索 Query，从 Case 库中召回相似历史案例并提取可复用信息，进而生成方案建议。对于高匹配案例提供修复和复用建议，部分匹配案例提供验证与排查建议，无可靠案例或证据不足时主动降级并提示人工介入。最终，建议主要来自部门已经结案的真实问题，使输出更符合内部代码和工程规范，也提高了方案的可追溯性和可复用性。随着新工单不断结案，经过工程师确认的现象、根因、修复和验证结果会继续结构化入库，形成历史 Case 数据闭环。



3、Bag 数据

车辆运行或台架测试过程中采集的多源时序数据，包含车辆状态、传感器信息、控制信号和模块输出等，可以对问题发生时的行驶状态和模块数据流进行回放。

4、Tree-sitter

主要用于对代码进行语法解析，按语法结构切分代码而不是简单按照固定大小切分。通过构建语法树可以识别函数、类、变量以及调用关系，使代码块尽量保持语义完整。

5、Neo4j

主要存储代码实体之间的关系。图数据库可以直接执行多跳关系查询，更适合调用链扩展。

6、Planning

Plan → Intent recognition (意图识别) → Task Decomposition (任务分解) → Act → Replan

7、Tools

Jira、Data、Code、Case

8、Memory

我的项目中记忆管理分为短期记忆和长期记忆。短期记忆对应单个工单的工作记忆，主要保存分类结果、数据和代码证据、根因候选、工具调用结果以及执行过程中的中间状态，通过 LangGraph 的 State 和 Checkpoint 实现，用于保持当前任务的上下文连续性。但由于上下文窗口容量有限，这里我

参考了 DeepSeek Harness 的 Compaction 机制：当对话历史过长时，完整的工作状态仍保存在 State 和 Checkpoint 中，只将较早的对话历史压缩成摘要并保留近期上下文，压缩后上下文变为当前工作状态加摘要加最近原文的结构。长期记忆进一步拆分为语义记忆、情景记忆和程序性记忆三类，用于跨工单、跨会话的保存复用信息。语义记忆主要保存分类体系、模块依赖、数据 Schema、版本映射等领域知识；情景记忆保存经过工程师确认的历史 Case，作为根因分析和方案建议阶段的历史经验；程序性记忆则保存 Skills，用于指导不同类型故障的处理流程。

9、Agent 评估

我采用 EDD 驱动整个项目生命周期，首先定义评测指标，并标注200多条历史工单作为离线 Benchmark，基于该评测集完成模型选型。随后进行评测，评测主要分为三层。第一层是组件评测，也就是我们的业务层，Triage 阶段计算分类后的 Accuracy，Code 和 Case RAG 依次计算 Recall@K、Precision@K、Hit Rate@K 和 MRR，工具调用逐项检查工具名称、参数 Schema 和执行结果。Solution 阶段是开放任务，在历史 Case 的基础上结合 LLM-as-Judge 来评测，这里遵循能用确定性规则判分就不用 LLM Judge。第二层是轨迹评测，评估 Multi-Agent 协作是否正确。工单处理完后，将实际 Trace 与标准链路比较，避免最终根因恰好正确但中间过程错误。第三层是端到端评测，主要关注整条链路的性能和资源消耗。完整输入一张工单，规则模式总耗时大概是 30 秒，Token 消耗 35K 左右，如果发生 Replan，时间和 Token 开销还会增加。

10、Harness Engineering

是围绕模型构建运行环境、调度机制和管控基础设施的一种工程实践，通过外部记忆、任务拆解、测试门禁、状态交接和人工审核，把模型的执行过程工程化，让 Agent 在复杂的真实任务中稳定高效地运行，业内比较认可的一个定义是 Agent = Model + Harness。

如果要用 Harness 工程实现一个项目，怎么设计？

我会从一个 Initializer + Coding Loop 的最小闭环开始，而不是一开始就做复杂的多 Agent 系统，核心思路是先拆解，再循环执行，靠外部状态保持连续性，靠确定性验证保证质量。

首先，我会准备一份清晰的 Spec / PRD，把产品目标、功能范围和验收标准定义清楚。因为在 Anthropic 的架构里，系统起点不是直接写代码，而是先把需求整理成后续 Agent 可以反复执行的任务。

然后，我会实现一个 Initializer Agent，它的职责不是写业务代码，而是读需求文档，把长任务拆成一组细粒度、可执行验证的功能，并生成功能列表等相应的状态文件。同时它还会初始化项目骨架、代码仓库，以及后续执行循环所需的基础环境，这样系统就有了任务清单和状态来源。

接着，我会实现一个 Coding Loop，每一轮都启动一个全新的 Coding Agent，让它读取当前状态文件、进度记录和代码库信息，只完成一个明确的小任务（任务原子化）。任务完成后，不是让 Agent 自己宣布成功，而是必须经过测试、linter 或类型检查等确定性验证，只有通过验证，才更新状态文件、写入交接记录并提交代码（验证优先）。随后这个 Agent 立即退出，下一轮由新的 Agent 接力（上下文刷新）。

此外，我会把外部记忆做成系统核心，而不是靠模型上下文来保持连续性。具体来讲，我会依赖功能列表、进度文件、Git 日志这些可持久化信息，让每个新 Agent 启动时都能快速接手，而不是依赖前一轮 session 的记忆（状态外部化）。

最后，我会在关键节点加入 Human-in-the-loop。在高风险功能、关键界面变更或多轮失败后，设计人工确认断点，确保系统不会沿着错误方向持续推进。

11、Loop Engineering

是让 Agent 围绕一个目标持续行动，由外部验证器检查结果，失败就回到循环，达标或触发硬停止条件才结束。它和定时任务（Cron）最本质的区别在于一决策者是否在循环内部。Cron 跑的是固定脚本，步骤永远不变，而 Loop 跑的是一个 Agent，它会观察当前状态、自己决定下一步、执行、检查结果，再判断要不要继续、重试还是停下。一个完整的 Loop 工程由定时任务、Worktree、Skill、连接器（MCP）、子 Agent、状态文件等部分组成。定时任务设定项目多久自动跑一次，Worktree 给每个 Agent 一个独立分支，所有改动都在各自的沙盒里，不碰主分支，状态文件是外部的公共记

忆。/goal、/loop 是 Claude Code 写好的标准 loop，/goal 执行给定的目标直至完成，/loop 到点重跑，不判断是否完成。

12、Graph Engineering

是在 Loop 之上再加一层编排，把复杂的 Agent 工作过程显式画成一张执行图：节点承担任务，边决定下一步运行什么，系统可以分支、并行、汇聚、验证、返工或停止。它和普通流水线的区别在于，流水线的步骤遵循固定规则，而 Graph 里的 Agent 会去解释任务，节点是概率性的，可能误解指令，也可能下一次运行做出不同选择。